Ground Ops: Safety & Compliance Engine

Code

Full-Stack Data Analysis and Data Engineering: Python Ingestion, SQL ETL, and R Automated Alerting System

AUTHOR
David Namgung

PUBLISHED
April 8, 2026

LinkedIn PORTFOLIO GitHub

PYTHON POSTGRESQL R (TIDYVERSE) DATA DICTIONARY ETL PIPELINES AUTOMATED ALERTING DATA QUALITY (DQ)

Launch Interactive Ops Dashboard

Download Report as a PDF

Executive Summary

In global aviation, ground operations (turnarounds) are high-risk environments where minutes of delay translate to millions in losses, and safety errors translate to lives.

This project simulates a **Production Data Pipeline** for ground audit logs. I engineered a system that ingests **100,000 raw flight records**, cleanses them via a **SQL Silver-Layer**, and utilizes an **R-based Validation Engine** to trigger automated safety alerts when airport performance drifts outside of regulatory thresholds.

Project Architecture

The engine is built on a modular “Bronze-to-Gold” architecture:

1. **Ingestion (Python/SQLAlchemy):** Synthetic generation of 100k audit logs with injected “noisy” data (outliers, missing values, typos).

2. **Transformation (PostgreSQL):** A multi-stage SQL pipeline standardizing station codes, resolving entity naming conflicts, and calculating performance variances.
3. **Validation (R/Dplyr):** A statistical watchdog script that monitors incident rates and flags high-risk stations for immediate audit.
4. **Intelligence (Tableau):** An executive-facing dashboard for real-time situational awareness.

---

1. Data Ingestion & Raw Loading

The first challenge was simulating “dirty” data consistent with real-world sensor logs. Using **Python**, I generated a dataset where 2% of flights contained safety incidents and several thousand records contained “impossible” physical values (negative baggage delays) to test the robustness of the pipeline.

Python Implementation

```
# Extract from 01_generate_and_load.py
import pandas as pd
from sqlalchemy import create_engine

# Database Connection
engine = create_engine('postgresql://davidnamgung:@localhost:5432/groundops')

# Pushing 100k records to PostgreSQL 'Raw' table
df.to_sql('raw_audit_logs', engine, if_exists='replace', index=False)
```

2. Data Governance & Schema

To maintain transparency across the pipeline, the following data dictionary defines the attributes for both the raw ingestion and the engineered silver layer.

Raw Audit Logs | Staged Audit Logs

The initial landing zone for raw sensor data and manual audit entries.

Column	Type	Description
`audit_id`	UUID	Primary Key. Unique identifier for the ground audit.
`flight_date`	Date	The date of the ground operation.

Column	Type	Description
station_code	String	3-letter IATA code. Contains raw noise (typos/spaces).
handler_name	String	The ground handling vendor (e.g., Menzies, dnata, Swissport).
turnaround_target_mins	Int	The IATA-standard scheduled turnaround time.
turnaround_actual_mins	Int	The observed duration of the ground operation.
baggage_delay_mins	Int	Raw delay minutes. May contain negative sensor noise.
safety_incident_flag	Bool	Binary indicator of a safety protocol breach.

3. SQL ETL and Data Cleansing

After initial data profiling, I utilized Common Table Expressions (CTEs) to transform raw logs into an analysis-ready state and performed a Data Quality audit which identified 1250 records with missing turnaround data. This strict filtering logic excludes incomplete records, ensuring a 100% complete dataset of flight logs. Key operations included:

Key Operations | SQL Implementation

- Feature Engineering: Turnaround variance and on-time status.
- String Normalization: Trimming and casing codes (e.g., yul → YUL).
- Entity Resolution: Merging duplicate vendor names (Menzies Aviation → Menzies).
- Outlier Handling: Nullifying turnaround times exceeding 500 minutes to prevent skewing KPIs.

4. Exploratory Analysis

4.1 The Performance Spread (Univariate)

Description | SQL Implementation | Results

In aviation, averages are deceptive. A station might have an average delay of zero minutes because one flight was 30 minutes early and another was 30 minutes late. This query analyzes the Distribution of Variance to identify our true operational risk: the 'Fat Tail' of extreme delays. Understanding this spread allows operations to move away from mean-based planning and focus on mitigating cascading delays.

4.2 The Speed-Safety Tradeoff (Bivariate)

Description | SQL Implementation | Results

A core hypothesis in ground operations is that rushing creates risk. This query tests that hypothesis by cross-referencing Schedule Compliance (On-Time vs. Delayed) with Safety Protocol Breaches. By isolating flights that beat their target turnaround time, we can determine if the pressure to hit SLA (Service Level Agreement) targets is inadvertently incentivizing unsafe behavior on the tarmac.

As turnaround times get shorter (closer to 0), the probability of a safety breach increases.

$$\rho(\text{Variance}, \text{Incident Rate}) < 0$$

4.3 The Vendor Performance Matrix

Description | SQL Implementation | Results

Airline operations rely heavily on third-party vendors for baggage handling, fueling, and dispatch. This query creates a Vendor Accountability Matrix. By aggregating performance metrics by handler_name, we move beyond anecdotal complaints and establish a data-backed baseline for contract renewals, allowing us to identify which vendors prioritize speed and which prioritize safety.

4.4 Station Risk Profiling

Description | SQL Implementation | Results

Safety risks are not distributed equally across the globe. Infrastructure bottlenecks, local weather, and regional staffing levels all impact performance. This query generates a Station Risk Profile, calculating the critical incident rate for each IATA hub. This intelligence allows the central operations team to intelligently deploy safety auditors to the highest-risk airports rather than relying on random inspections.

5. Automated Validation Engine in R

A dashboard is passive; safety requires active alerting. I developed an R script that acts as an automated auditor. It calculates the safety incident rate per station and compares it against a Critical Threshold (2.5%).

The incident rate is calculated as:

$$Incident\ Rate = \frac{\sum Safety\ Incidents}{\sum Total\ Flights} \times 100$$

If any airport (e.g., YUL, LHR) breaches this threshold, the system triggers a simulated high-priority alert.

The audit cleans data, calculates failure rates, and triggers simulated alerts.

```
library(dplyr)
library(DBI)
library(RPostgres)

print("Connecting to the 'groundops' database...")
con <- dbConnect(RPostgres::Postgres(),
                 dbname = "groundops",
                 host = "localhost",
                 port = 5432,
                 user = "davidnamgung", # Your local Mac username
                 password = "")

staged_data <- dbReadTable(con, "staged_audit_logs")
dbDisconnect(con)

CRITICAL_THRESHOLD_PCT <- 2.5

print("Running compliance audit across all global stations...")

audit_results <- staged_data %>%
  group_by(station_code) %>%
  summarize(
    total_flights_handled = n(),
    total_safety_incidents = sum(safety_incident_flag == TRUE, na.rm = TRUE),
    incident_rate_pct = round((total_safety_incidents / total_flights_handled) * 100, 2)
```

```
  ) %>%
  arrange(desc(incident_rate_pct))

failing_stations <- audit_results %>% filter(incident_rate_pct > CRITICAL_THRESHOLD_PCT)

if (nrow(failing_stations) > 0) {
  cat("⚠️ CRITICAL ALERT: The following stations have breached the safety threshold:\n\n"
  for (i in 1:nrow(failing_stations)) {
    cat(sprintf("   -> STATION: %s | Incident Rate: %.2f%% (Threshold: %.2f%%)\n",
                failing_stations$station_code[i],
                failing_stations$incident_rate_pct[i],
                CRITICAL_THRESHOLD_PCT))
  }

  cat("\nACTION REQUIRED: Dispatch audit team to failing stations immediately.\n")
} else {
  cat("✅ ALL CLEAR: All stations are operating within acceptable safety thresholds.\n")
}
cat("==================================================\n\n")
```

6. Technical Outcomes

Data Integrity: Eliminated 100% of negative baggage delays and standardized 7 disparate airport code formats.

Operational Insight: Identified specific handlers (e.g., “Menzies”) responsible for the highest turnaround variance.

Efficiency: Designed a pipeline capable of processing 100,000 rows from raw-to-alert in under 30 seconds.

7. Forward Outlook: Enterprise Production Architecture

While this portfolio project executes via a localized batch-processing methodology, it was architected to mirror the logic of a modern **Enterprise Data Stack**. In a real-world Ground Operations scenario, the reliance on flat files (.csv) and manual script execution is eliminated to ensure real-time situational awareness.

If deployed within an airline’s actual tech stack, the pipeline would be scaled using the following methodologies:

(Apache Airflow or cron)

Instead of executing Python, SQL, and R sequentially via a local terminal, the pipeline would be converted into a **Directed Acyclic Graph (DAG)** using Apache Airflow or Prefect.

- **The Benefit:** Airflow would trigger the ingestion script on an automated cron schedule (e.g., every 5 minutes), automatically handling retries if the airline's API experiences downtime.